

# Technology Requirements Document (TRD)

# Project Name

ChatVault

WhatsApp Export Chat Viewer & Media Player

---

## # 1. Technical Overview

ChatVault is a client-side web application designed to parse and render exported WhatsApp chat files in a WhatsApp-inspired read-only interface.

The application processes:

- TXT exports
- ZIP exports
- media attachments
- audio playback

All operations occur locally inside the browser without uploading chats to external servers.

---

## # 2. Core Technology Stack

### ## 2.1 Frontend Framework

| Technology   | Version       | Purpose        |
|--------------|---------------|----------------|
| React        | Latest Stable | UI Rendering   |
| Next.js      | Latest Stable | App Framework  |
| TypeScript   | Latest Stable | Static Typing  |
| Tailwind CSS | Latest Stable | Styling System |

---

### ## 2.2 Runtime Environment

| Technology | Purpose             |
|------------|---------------------|
| Node.js    | Development Runtime |
| npm/pnpm   | Package Management  |

---

### ## 2.3 State Management

| Technology | Purpose                  |
|------------|--------------------------|
| Zustand    | Lightweight Global State |

Reason:

- simpler than Redux
- faster setup
- minimal boilerplate
- better for media/chat state

---

### ## 2.4 Audio Playback Stack

| Technology | Purpose |
|------------|---------|
|------------|---------|

```
|---|---|
| HTML5 Audio API | Native Playback |
| WaveSurfer.js | Audio Waveforms |
| Web Audio API | Audio Processing |
```

---

## ## 2.5 File Processing Stack

```
| Technology | Purpose |
|---|---|
| JSZip | ZIP Extraction |
| FileReader API | File Reading |
| Blob API | Local File Handling |
```

---

## ## 2.6 Rendering Optimization

```
| Technology | Purpose |
|---|---|
| react-window | Virtualized Lists |
| Intersection Observer API | Lazy Media Loading |
```

---

## ## 2.7 Media Rendering

```
| Technology | Purpose |
|---|---|
| Native Browser Media APIs | Image/Video Rendering |
| react-player | Video Playback |
```

---

## # 3. Browser Requirements

### ## Supported Browsers

```
| Browser | Minimum Version |
|---|---|
| Chrome | Latest 2 versions |
| Edge | Latest 2 versions |
| Firefox | Latest 2 versions |
| Safari | Latest 2 versions |
```

---

### ## Unsupported Browsers

- Internet Explorer
- Legacy Android browsers

---

## # 4. Application Architecture

### ## 4.1 High-Level Architecture

User Upload

↓

ZIP/TXT Processor

↓

WhatsApp Parser Engine

```
↓
Media Resolver
↓
State Store
↓
Chat Renderer
↓
Audio/Media Engine
```

---

## ## 4.2 Frontend Architecture

```
Presentation Layer
↓
State Layer
↓
Parser Layer
↓
Media Layer
↓
Storage Layer
```

---

## # 5. Folder Structure

```
src/
├── app/
├── components/
│   ├── chat/
│   ├── audio/
│   ├── media/
│   ├── layout/
│   └── common/
├── parser/
├── services/
├── hooks/
├── stores/
├── types/
├── utils/
├── styles/
└── constants/
```

---

## # 6. Parser Engine Requirements

### ## 6.1 Supported Formats

| Format    | Support  |
|-----------|----------|
| .txt      | Required |
| .zip      | Required |
| .opus     | Required |
| .mp3      | Required |
| .m4a      | Required |
| .jpg/.png | Required |
| .mp4      | Required |
| .pdf/.doc | Optional |

---

### ## 6.2 Parser Functionalities

The parser must:

- detect timestamps
- detect sender names
- detect message content
- detect media references
- detect deleted messages
- detect system messages

---

### ## 6.3 Parsing Strategy

Use:

- streaming parsing
- chunk-based processing

Avoid:

- loading full large files into memory

---

### ## 6.4 Message Regex Pattern

```
^\[(.+?)\]\s(.+?):\s(.+)\$
```

---

## # 7. Internal Data Models

### ## 7.1 Chat Message Interface

```
interface ChatMessage {  
    id: string;  
    sender: string;  
    timestamp: number;  
    type: MessageType;  
    content: string;  
    mediaPath?: string;  
    duration?: number;  
    isMine: boolean;  
}
```

---

### ## 7.2 Message Types

```
type MessageType =  
    | "text"  
    | "audio"  
    | "image"  
    | "video"  
    | "document"  
    | "system";
```

---

## # 8. UI Rendering Requirements

### ## 8.1 Chat Interface

The UI must support:

- left/right message alignment
- grouped bubbles

- timestamps
- sender labels
- date separators
- media cards
- responsive layouts

---

## ## 8.2 Virtual Rendering

The app must:

- render only visible messages
- support 100k+ messages
- maintain smooth 60 FPS scrolling

---

## ## 8.3 Responsive Design

Desktop:

Sidebar + Chat View

Mobile:

Full-width Chat Screen

---

## # 9. Audio System Requirements

### ## 9.1 Playback Features

Required:

- play/pause
- seek
- waveform
- progress tracking
- playback speed
- background scrolling playback

---

### ## 9.2 Playback Constraints

Only one audio instance can play simultaneously.

---

### ## 9.3 Audio State Management

Use singleton audio controller:

```
class AudioManager {}
```

---

## # 10. Storage Requirements

### ## 10.1 Local Storage

Use:

- IndexedDB

Purpose:

- cache parsed chats

- improve reload speed
- offline persistence

---

## ## 10.2 Cache Policy

|                      |           |
|----------------------|-----------|
| Data                 | Cached    |
| ---                  | ---       |
| Parsed Messages      | Yes       |
| Media Metadata       | Yes       |
| Uploaded Media Files | Temporary |

---

## # 11. Security Requirements

### ## 11.1 Privacy

The application must:

- never upload chats automatically
- process locally only
- avoid server-side parsing

---

### ## 11.2 XSS Protection

All rendered messages must:

- sanitize HTML
- escape injected scripts

---

### ## 11.3 File Validation

Validate:

- MIME types
- extension types
- corrupted ZIPs

---

## # 12. Performance Requirements

|                 |           |
|-----------------|-----------|
| Metric          | Target    |
| ---             | ---       |
| Initial Parse   | < 5 sec   |
| Scroll FPS      | 60 FPS    |
| Audio Latency   | < 150ms   |
| Memory Usage    | optimized |
| Lazy Load Delay | < 300ms   |

---

## # 13. Scalability Requirements

The system must support:

- 100k+ messages
- chats larger than 500MB
- thousands of media files

---

## # 14. Error Handling Requirements

| Error             | Handling          |
|-------------------|-------------------|
| Missing TXT       | show upload error |
| Missing Media     | placeholder UI    |
| Corrupted ZIP     | graceful fail     |
| Unsupported Media | fallback renderer |

---

## # 15. Accessibility Requirements

Support:

- keyboard navigation
- screen readers
- responsive text scaling
- high contrast mode

---

## # 16. Deployment Requirements

### ## Hosting Platform

Recommended:

- Vercel

Alternative:

- Netlify

---

## # 17. CI/CD Requirements

Use:

- GitHub Actions
- automated builds
- lint checks
- type checks

---

## # 18. Development Environment

| Tool     | Purpose    |
|----------|------------|
| VS Code  | IDE        |
| ESLint   | Linting    |
| Prettier | Formatting |
| Husky    | Git Hooks  |

---

## # 19. Recommended Libraries

| Feature        | Library       |
|----------------|---------------|
| ZIP Extraction | JSZip         |
| Waveform       | WaveSurfer.js |
| Virtual List   | react-window  |
| State          | Zustand       |
| Dates          | dayjs         |
| Icons          | Lucide React  |

---

## # 20. Future Technical Enhancements

Future upgrades may include:

- WebAssembly parser
- AI summarization
- OCR media extraction
- sentiment analysis
- cloud sync
- PWA support
- Electron desktop app